MTRC

MODERN INSTITUTE OF TECHNOLOGY AND RESEARCH CENTRE

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Faculty Name	Sanjeev Khatri		
Subject Name with Code	4AD4-06 Introduction to Java Programming		
Semester	4th	**Session**	2022-23 (Even)
Branch	AIDS	**Batch**	A
Unit No.	2	**Submission Date**	15-Mar-23

Lecture No.	*Topic Covered*	*Page No.*	*Total Pages*
1	Class fundamentals, declaring class, creating objects	1 - 3	3
2	Introducing methods: method declaration, overloading	4 - 6	3
3	Using objects as parameters	7 - 7	1
4	Recursion, Constructors	8 - 10	3
5	This keyword	11 - 13	3
6	Garbage collection, the finalization	14 -15	2

Faculty HOD Dean Academics 15/3/2023

UNIT-2 (INTRODUCTION TO CLASSES)

LECTURE - 1

Class Fundamentals, Declaring Class, Creating Objects

Class Fundamentals

A class is a template that defines the form of an object. It specifies both the data and the code that will operate on that data. Java uses a class specification to construct objects. The concept of a class is useful because of its reusability.

Objects

Objects are instances of a class. Every object that is an instance of a class is assumed to possess the same methods as defined in a class.Objects gain their attributes from their classes, and all objects have predictable attributes because they are members of certain classes.

Declaring a Class

When we define a class, we declare its exact form and nature. We do this by specifying the instance variables that it contains and the methods that operate on them. Although very simple classes might contain only methods or only instance variables, most realworld classes contain both.

A class is created by using the keyword class. A simplified general form of a class definition is as under:

```
class classname
{
        // declare instance variables
        type var1;
        type var2;
        // ...
        type varN;
```


MITRC

```
        // declare methods
        type method1(parameters)
        {
                // body of method
        }
        type method2(parameters)
        {
                // body of method
        }
        // ...
        type methodN(parameters)
        {
                // body of method
        }
}
```

Example-

```
class calc
{
    int sum(int a, int b)
    {
        int c = a + b;
        return c;
    }
    int sub(int a, int b)
    {
        int c = a - b;
        return c;
    }
}
```

MITRC

Creating an Object

A two-step process creates an object that is an instance of a class. First, we supply a type and an identifier—just as when we declare any variable—and then we allocate computer memory for that object. The general format of creating an object is-

ClassName VariableName = new ClassName();

The **new operator** dynamically allocates (that is, allocates at run time) memory for an object and returns a reference to it. This reference is, essentially, the address in memory of the object allocated by new. This reference is then stored in a variable. Thus, in Java, all class objects must be dynamically allocated.

In above example we have created a class calc having two methods – sum and sub. Now the following program is describing how we can create object for that class-

```
public class classobjexp1 {
    public static void main(String abc[])
    {
        calc cl = new calc();
        int vl1=5;
        int vl2=6;
        int vl3 = cl.sum(vl1, vl2);
        System.out.println("Sum of Two Values is : " + vl3);

    }
}

class calc
{
    int sum(int a, int b)
    {
        int c = a + b;
        return c;
    }
}
```

LECTURE - 2

Introducing Methods : Method Declaration, Overloading

Methods

A method is a program module that contains a series of statements that carry out a task. As we have already seen Java classes that contain a main() method, which executes automatically when we run a program. A program's main() method can execute additional methods, and those methods can execute others.

To execute a method, we invoke or call it. In other words, a calling method makes a method call, and the method call invokes a called method. The calling method is also known as a client method because a called method provides a service for its client.

Method Declaration

Every method must include the two parts-

Method Header—A method's header provides information about how other methods can interact with it. A method header is also called a declaration.

The method header is the first line of a method. It contains the following:

- Optional access specifiers - Public, Private or Protected
- A return type - The data type of the value to be return
- An identifier - A variable name which helps to identify the method
- Parentheses - Parentheses may contain parameters.

Method Body - The method body contains the statements between a pair of curly braces that carry out the work of the method.

Static - Any method that can be used with or without instantiating an object requires the keyword modifier **static**.

Example - Program having two methods with keyword static-

```
public class methodexp1 {
    public static void main(String abc[])
    {
        int vl1=5;
        int vl2=6;
        int vl3 = sum(vl1, vl2);
```


MITRC

```
        System.out.println("Sum of Two Values is : " + vl3);
    }

    public static int sum(int a, int b)
    {
        int c = a + b;
        return c;
    }

    public static int sub(int a, int b)
    {
        int c = a - b;
        return c;
    }

}
```

Example - Program having two methods without keyword static-

```
public class methodexp1 {
    public static void main(String abc[])
    {
        calc cl = new calc();
        int vl1=5;
        int vl2=6;
        int vl3 = cl.sum(vl1, vl2);
        System.out.println("Sum of Two Values is : " + vl3);
        vl3 = cl.sub(vl1, vl2);
        System.out.println("Subtraction of Two Values is : " + vl3);
    }
}

class calc
{
    public int sum(int a, int b)
    {
        int c = a + b;
        return c;
    }

    public int sub(int a, int b)
    {
        int c = a - b;
        return c;
    }
}
```

Method Overloading

In Java we can define two or more methods within the same class that share the same name, as long as their parameter declarations are different. When this is the case, the methods are said to be overloaded, and the process is referred to as method overloading. Method overloading is one of the ways that Java implements polymorphism.

MITRC

LECTURE - 3

Using Objects as parameters

A method can take an objects as a parameter. When we pass an object to a method, the situation changes dramatically, because objects are passed by what is effectively call-by-reference.

Example-

```
public class objasparameter
{

    public static void main(String abc[])
    {
        calc cl = new calc();
        int vl1=5;
        int vl2=6;
        show(cl, vl1, vl2);
    }

    public static void show(calc clc, int vl1, int vl2)
    {
        int vl3 = clc.sum(vl1, vl2);
        System.out.println("Sum of Two Values is : " + vl3);
        vl3 = clc.sub(vl1, vl2);
        System.out.println("Subtraction of Two Values is : " + vl3);
    }
}


class calc
{
    public int sum(int a, int b)
    {
        int c = a + b;
        return c;
    }

    public int sub(int a, int b)
    {
        int c = a - b;
        return c;
    }
}
```

LECTURE - 4

Recursion, Constructors

Recursion

When a method calls itself, this process is called recursion, and a method that calls itself is said to be recursive. In general, recursion is the process of defining something in terms of itself and is somewhat similar to a circular definition. Recursion is a powerful control mechanism.

Example-

```
class Factorial
{
    int factR(int n)
    {
        int result;
        if(n==1)
        {
            return 1;
        }
        result = factR(n-1) * n; //Use of Recursion by calling the same method
within the method
        return result;
    }

    int factI(int n)
    {
        int t, result;
        result = 1;
        for(t=1; t<=n;t++)
        {
            result *= t;
        }
        return result;
    }
}

class recursionexp1
{
    public static void main(String args[])
    {
        Factorial f = new Factorial();
        System.out.println("Factorials using recursive method.");
        System.out.println("Factorial of 3 is " + f.factR(3));
        System.out.println("Factorial of 4 is " + f.factR(4));
```

```
        System.out.println("Factorial of 5 is " + f.factR(5));

        System.out.println("Factorials using iterative method.");
        System.out.println("Factorial of 3 is " + f.factI(3));
        System.out.println("Factorial of 4 is " + f.factI(4));
        System.out.println("Factorial of 5 is " + f.factI(5));

    }
}
```

Constructor

When we create a method by naming it's class name, it is called a constructor. When we don't write a constructor for a class, Java writes one for us. A constructor defines what occurs when an object of a class is created. Constructors are an important part of all classes and have many significant attributes. Most real-world classes explicitly define their own constructors within their class definition.

```
public class constexp1 {
    public static void main(String abc[])
    {
        int vl1=5;
        int vl2=6;
        calc cl = new calc(vl1, vl2);
        int vl3 = cl.sum();
        System.out.println("Sum of Two Values is : " + vl3);
        int vl4 = cl.sub();
        System.out.println("Subtraction of Two Values is : " + vl4);
    }
}

class calc
{
    int a;
    int b;
    calc(int a1, int b1)
    {
        a = a1;
        b = b1;
    }

    int sum()
    {
        int c = a + b;
        a = c;
        return c;
    }
```

```
    int sub()
    {
        int c = a - b;
        return c;
    }
}
```

Output-
Sum of Two Values is : 11
Subtraction of Two Values is : 5

MITRC

LECTURE - 5

This Keyword

Sometimes a method will need to refer to the object that invoked it. To allow this, Java defines the **this** keyword. **This** can be used inside any method to refer to the current object. That is, **this** is always a reference to the object on which the method was invoked.

Example (Without this)-

```
public class thisexp1 {
    public static void main(String args[])
    {
        Pwr x = new Pwr(6.0, 3);
        Pwr y = new Pwr(2.5, 1);
        Pwr z = new Pwr(5.7, 0);

        System.out.println(x.b + " raised to the " + x.e +
            " power is " + x.get_pwr());
        System.out.println(y.b + " raised to the " + y.e +
            " power is " + y.get_pwr());
        System.out.println(z.b + " raised to the " + z.e +
            " power is " + z.get_pwr());
    }
}

class Pwr
{
    double b;
    int e;
    double val;

    Pwr(double base, int exp)
    {
        b = base;
        e = exp;

        val = 1;
        if(exp == 0)
        {
            return;
        }

        for(;exp>0;exp--)
        {
            val = val * base;
        }
```

```
    }

    double get_pwr()
    {
        return val;
    }
}
```

Example (With this)-

```
public class thisexp1 {
    public static void main(String args[])
    {
        Pwr x = new Pwr(6.0, 3);
        Pwr y = new Pwr(2.5, 1);
        Pwr z = new Pwr(5.7, 0);

        System.out.println(x.b + " raised to the " + x.e +
            " power is " + x.get_pwr());
        System.out.println(y.b + " raised to the " + y.e +
            " power is " + y.get_pwr());
        System.out.println(z.b + " raised to the " + z.e +
            " power is " + z.get_pwr());
    }
}

class Pwr
{
    double b;
    int e;
    double val;

    Pwr(double base, int exp)
    {
        this.b = base;
        this.e = exp;

        this.val = 1;
        if(exp == 0)
        {
            return;
        }

        for(;exp>0;exp--)
        {
            this.val = this.val * base;
        }
    }
```

```
    double get_pwr()
    {
        return this.val;
    }
}
```

Output (without this)-
6.0 raised to the 3 power is 216.0
2.5 raised to the 1 power is 2.5
5.7 raised to the 0 power is 1.0

Output (with this)-
6.0 raised to the 3 power is 216.0
2.5 raised to the 1 power is 2.5
5.7 raised to the 0 power is 1.0

As we see, both outputs are same, and nothing changed. However, **this** has some important uses. For example, the Java syntax permits the name of a parameter or a local variable to be the same as the name of an instance variable. When this happens, the local name hides the instance variable. We can gain access to the hidden instance variable by referring to it through this.

LECTURE - 6

Garbage Collection, The Finalization

Grabage Collection

Objects are dynamically allocated from a pool of free memory by using the **new** operator. As explained, memory is not infinite, and the free memory can be exhausted. Thus, it is possible for **new** to fail because there is insufficient free memory to create the desired object. For this reason, a key component of any dynamic allocation scheme is the recovery of free memory from unused objects, making that memory available for subsequent reallocation. To handle this situation Java uses a different, more troublefree approach: **garbage collection**.

Java's garbage collection system reclaims objects automatically occurring transparently, behind the scenes, without any programmer intervention. It works like this: when no references to an object exist, that object is assumed to be no longer needed, and the memory occupied by the object can be reclaimed.

In short, Garbage collection is the mechanism that recycles unused objects so that their memory can be reused.

Finalization

Just before destroying any object, the garbage collector always calls **finalize()** method to perform clean-up activities on that object. This process is known as Finalization in Java.

finalize() method in Java is a method of the Object class that is used to perform cleanup activity before destroying any object. It is called by Garbage collector before destroying the objects from memory. **finalize()** method is called by default for every object before its deletion.

Example-

```
public class finalizedemo {

  protected void finalize() throws Throwable {
    try {
      System.out.println("This is the finalize method of finalizedemo Class.");
    }
    catch (Throwable e) {
      throw e;
    }
    finally {
      System.out.println("Finalize method of the Object class is called");
```

```
      // Calling finalize() of Object class
      super.finalize();
    }

  }

  public static void main(String[] args) throws Throwable {
    // Creating demo's object
    finalizedemo d = new finalizedemo();

    // Calling finalize of demo
    d.finalize();
  }
}
```

Note- Finalization is deprecated and subject to removal in a future release. The use of finalization can lead to problems with security, performance, and reliability.

Final vs Finally vs finalize() in Java

Java has 3 different keywords used in different scenarios final, finally, and finalize. These are different keywords with completely different functionalities. Let's understand their uses and differences:

final	**finally**	**finalize**
final is a keyword and access modifier in Java.	finally, block is used in Java Exception Handling to execute the mandatory piece of code after try-catch blocks.	finalize() is the method in Java.
final access modifier is used to apply restrictions on the variables, methods, and classes.	finally block executes whether an exception occurs or not. It is primarily used to close resources.	finalize() method is used to perform clean-up processing just before an object is garbage collected.
It is used with variables, methods, and classes.	It is with the try-catch block in exception handling.	It is used with objects.
Once declared, the final variable becomes constant and can't be modified.	finally block cleans up all the resources used in the try block.	finalize method performs the cleaning with respect to the object before its destruction.
final is executed only when we call it.	finally block executes as soon as the execution of try-catch block is completed without depending on the exception.	finalize method is executed just before the object is destroyed.